

RELAZIONE HOMEWORK 3

GIOELE MESSINA – LUCA CAMIOLO

1. DESCRIZIONE DEL SISTEMA

Il progetto estende l'architettura sviluppata nei precedenti progetti introducendo un sistema di white-box monitoring tramite Prometheus di alcuni microservizi e la migrazione a k8s.

Per quanto riguarda il sistema di monitoraggio white-box è stato utilizzato prometheus, il quale recupera le metriche esposte dai microservizi `user_manager`, `alert_system`, `notifier_system` e `data_collector`.

Il cluster Kubernetes è stato creato utilizzando **KIND (Kubernetes IN Docker)**, una soluzione che permette di eseguire cluster Kubernetes completamente all'interno di container Docker, astruendo i nodi fisici tramite container. Il deployment del cluster è gestito tramite **file YAML**, che definiscono diversi oggetti Kubernetes necessari al corretto funzionamento dell'applicazione, come **Service**, **Deployment** e **PersistentVolumeClaim**.

2. SCELTE PROGETTUALI

Per il monitoraggio delle metriche con Prometheus è stato deciso di utilizzare due tipi di metriche, in particolare Gauge e Counter, sulla base della metrica misurata. Le metriche di tipo Gauge sono metriche che sono state utilizzate per monitorare i tempi di elaborazione, in particolare la metrica **`opensky_download_duration_seconds`** registra il tempo impiegato per scaricare le informazioni relative ai voli di interesse per un dato aeroporto, mentre la metrica **`alert_user_processing_duration_seconds`** misura il tempo impiegato per il processamento dei messaggi sul topic `alert_system` specificando tramite le relative label l'aeroporto e l'utente per cui sono state scaricate le informazioni.

Per quanto riguarda le metriche di tipo counter, quelle utilizzate sono: **`alerts_sent_total`**, **`user_registered_total`**, **`user_deleted_total`**, **`circuit_breaker_failures_total`** e **`email_sent_total`** le quali misurano valori cumulativi ideali per conteggiare eventi come registrazioni ed eliminazioni effettuate dagli utenti, successi o fallimenti di operazioni ed il numero di messaggi processati e inviati per notificare gli utenti.

Per il porting su Kubernetes si è scelto di adottare un'architettura composta da un nodo master e un unico nodo worker, incaricato della gestione di tutti i pod, a causa dell'elevato consumo di risorse necessario per supportare un numero maggiore di nodi worker. Considerando uno scenario iniziale in cui il sistema non è soggetto a carichi elevati, si è deciso di configurare una singola replica per ciascun pod, impostando il parametro `replicas`: 1.

Nel caso in cui, in futuro, si decidesse di effettuare uno scaling del sistema, basterebbe aumentare il valore del parametro e k8s si occuperebbe di gestire automaticamente la replicazione e il load balancing delle richieste tra le varie repliche.

3. DETTAGLI IMPLEMENTATIVI

- **K8S**

Per gestire le dipendenze tra i vari microservizi sono stati utilizzati gli init containers negli oggetti di deployments, i quali permettono di rispettare un determinato ordine di avvio.

Gli initContainers sono container che vengono eseguiti in modo sequenziale prima dei container principali di un pod e servono a preparare l'ambiente o a verificare che determinate condizioni siano soddisfatte, consentendo così l'avvio ordinato dei microservizi.

Per implementare questo meccanismo di attesa, è stata utilizzata un'immagine leggera come busybox, contenente uno script shell che sfrutta lo strumento di connessione netcat per verificare la disponibilità della porta associata al Service del Deployment da attendere.

Il controllo viene ripetuto ciclicamente fino a quando il servizio richiesto non diventa disponibile, garantendo così un avvio ordinato e sincronizzato dei microservizi.

Il pod user_manager attende che prima sia in esecuzione il pod che implementa lo user_db, il data_collector, invece, oltre ad attendere che sia eseguito prima il pod relativo al data_db aspetta anche che sia eseguito il broker Kafka. I pod dell>alert_system e del notifier_system devono attendere che il broker kafka sia eseguito prima di potersi avviare.

Per garantire che i dati non vengano persi a causa di crash dei pod è stato deciso di utilizzare i Persistent Volume Claims in entrambi i Data Base (user_db e data_db) in modo da garantire una certa persistenza dei dati.

- **PROMETHEUS**

Per acquisire le metriche esposte dai microservizi si è deciso di utilizzare Prometheus, il quale espone le metriche acquisite sulla porta 9090. La configurazione di quali microservizi monitorare e le relative porte da usare per il recupero delle metriche è definita nel manifest prometheus.yaml. Le metriche esposte dai microservizi vengono raccolte periodicamente ogni 15 secondi e possono essere visualizzate collegandosi a: <http://localhost:9090/>.

-USER MANAGER

Per esporre le metriche il microservizio user_manager apre un http server sulla porta 9102 il quale consente a Prometheus di recuperare periodicamente le metriche raccolte. Le metriche esposte dal microservizio sono definite nel file user_logic.py e sono:

- ❖ **USER_REGISTERED_TOTAL:** metrica di tipo contatore che conta il numero di iscrizioni tramite la route /register andate a buon fine.
- ❖ **USER_DELETED_TOTAL:** metrica di tipo contatore che conta il numero di utenti che eliminano il proprio account.

Si è deciso di monitorare queste metriche per tenere sotto controllo costantemente il numero di utenti che si iscrivono e si cancellano dal servizio, in modo da poter determinare il numero di utenti attivi registrati. Questo consente di valutare l'utilizzo effettivo del servizio, consentendo anche di prendere decisioni opportune relative a scalabilità, performance e gestione delle risorse sulla base dell'effettivo numero di utenti.

-DATA_COLLECTOR

Le metriche esposte dal data_collector sono disponibili per Prometheus sulla porta 8000 e sono state definite all'interno del file OpenSky.py, nella cartella data_collector. In particolare, per esporre le metriche il microservizio apre un http server sulla porta 8000 consentendo a Prometheus di recuperare periodicamente le metriche raccolte dal microservizio. Le metriche esposte dal microservizio sono:

- ❖ **OPENSKY_DOWNLOAD_DURATION_SECONDS**: metrica di tipo gauge che viene aggiornata a ogni ciclo di interrogazione dell'API di OpenSky per recuperare le info dei voli e misura la durata complessiva dell'operazione di download e salvataggio nel DB delle informazioni. In particolare, per la metrica è stata definita una label airport che conterrà il codice ICAO dell'aeroporto per il quale si stanno scaricando le informazioni, permettendo di distinguere il tempo di risposta dell'API in base all'aeroporto analizzato.
- ❖ **CIRCUIT_BREAKER_FAILURES_TOTAL**: metrica di tipo *Counter* incrementata ogni volta che una richiesta di una chiamata API viene bloccata o fallisce in seguito al sollevamento di un'eccezione gestita dal Circuit Breaker. È stata definita per la metrica una label error_type, che indica la tipologia di errore che ha causato l'incremento del contatore (ad esempio apertura del Circuit Breaker o errore generico), consentendo di distinguere le diverse cause di fallimento delle richieste.

È stato deciso di catturare queste metriche in quanto la metrica DOWNLOAD_DURATION permette di monitorare le performance dell'API per il recupero delle informazioni con il relativo salvataggio nel DB, consentendo di capire se il sistema esterno sta lavorando come previsto o se ci sono anomalie in termini di latenza per particolari aeroporti. La metrica CB_FAILURES, invece, permette di monitorare quante richieste all'API sono state bloccate o fallite a causa del Circuit Breaker o di errori gestiti, permettendo di distinguere tra i vari tipi di fallimento.

-ALERT SYSTEM

Per esporre le metriche è stato aperto un http server sulla porta 8001 nel microservizio alert_system, il quale consente a Prometheus di recuperare periodicamente le metriche raccolte. Le metriche esposte dal microservizio sono definite nel file app.py nella cartella alert_system e sono:

- ❖ **ALERTS_SENT_TOTAL**: metrica di tipo Counter che conta il numero di messaggi inviati sul topic Kafka notifier_system. La metrica possiede una label

che viene utilizzata per specificare l'email dell'utente per cui è stato generato il messaggio da mandare sul topic `notifier_system`.

- ❖ **ALERT_USER_PROCESSING_TIME:** metrica di tipo gauge che misura il tempo necessario al microservizio `alert_system` per elaborare la logica di generazione di un'allerta per un singolo utente. Il tempo misurato comprende la valutazione del numero totale di voli per uno specifico aeroporto, il confronto con le soglie configurate dall'utente e l'eventuale invio della notifica al sistema di notifica tramite Kafka.

La metrica è etichettata per servizio, nodo, aeroporto ed e-mail dell'utente, consentendo un'analisi dettagliata delle prestazioni del sistema in funzione degli utenti e degli aeroporti monitorati.

La raccolta delle metriche **ALERTS_SENT_TOTAL** e **ALERT_USER_PROCESSING_TIME** consente di monitorare sia il corretto funzionamento del sistema di allerta sia le sue prestazioni operative. In particolare, il contatore permette di verificare il numero di notifiche generate verso il topic `to_notifier`, mentre la gauge consente di analizzare i tempi di elaborazione dei vari alert, individuando eventuali rallentamenti o colli di bottiglia legati a specifici utenti o aeroporti

-NOTIFIER SYSTEM

Per esporre la metrica è stato aperto un http server sulla porta 8002 nel microservizio `notifier_system`, il quale consente a Prometheus di recuperare periodicamente la metrica misurata. La metrica esposta dal microservizio è definita nel file `app.py` nella cartella `notifier_system`:

- ❖ **EMAIL_SENT_TOTAL:** metrica di tipo counter che conta il numero di e-mail notificate a ciascun utente.

Questa metrica, insieme a quella dell'`alert_system` *alerts_sent_total*, consente di verificare che tutti i messaggi pubblicati sul topic Kafka `to-notifier` vengano effettivamente tradotti in e-mail inviate ai rispettivi utenti, permettendo inoltre di monitorare il numero di e-mail recapitate a ciascun utente

4. MODIFICHE EFFETTUATE

Avendo effettuato il porting su K8s rispetto alla precedente implementazione il file `.env` è stato sostituito con il file `secret.yaml`, in cui si dovranno andare a specificare le credenziali di Open Sky, da utilizzare durante le query, ed i parametri per l'invio di e-mail tramite gmail.

5. TEST APPLICAZIONE

Rispetto alle precedenti versioni in cui l'esecuzione avveniva tramite comandi "docker compose", adesso con il passaggio a k8s è necessario innanzitutto creare il cluster (worker+master), successivamente montare le immagini dei microservizi e poi caricarle sul cluster ed infine applicare i manifest. Per semplicità e per automatizzare il tutto è stato

realizzato un file **commands.bat** che una volta eseguito eseguirà tutti i comandi in modo automatico. Una volta che tutti i pod sono in esecuzione (è possibile verificare con il comando `kubectl get pods -n flights-monitoring`) sarà possibile inviare le richieste tramite postman.